

Formalizacja teorii grup w Rocq

Aniela Dąbrowska, Jarosław Piórkowski

1 Wstęp

Celem projektu jest formalizacja podstawowych pojęć teorii grup z wykorzystaniem systemu dowodowego Rocq. W ramach pracy wprowadzono niezbędne definicje oraz udowodniono podstawowe twierdzenia i własności. Ponadto przedstawiono przykłady grup i podgrup wraz z formalnymi dowodami ich poprawności. Cały kod źródłowy znajduje się w jednym pliku `Groups.v`.

2 Opis teoretyczny

Teoria grup jest jedną z podstawowych struktur algebraicznych w matematyce. Grupy stanowią formalny sposób opisu zbiorów wyposażonych w działanie spełniające określone aksjomaty, co ma fundamentalne znaczenie w wielu działach matematyki, takich jak algebra, teoria liczb czy geometria.

2.1 Definicja grupy

Grupą nazywamy parę $(G, *)$, gdzie G jest niepustym zbiorem, a $*$: $G \times G \rightarrow G$ jest działaniem binarnym spełniającym następujące warunki:

- **Łączność:** dla wszystkich $a, b, c \in G$ zachodzi

$$(a * b) * c = a * (b * c).$$

- **Element neutralny:** istnieje element $e \in G$, taki że dla każdego $a \in G$:

$$e * a = a * e = a.$$

- **Element odwrotny:** dla każdego $a \in G$ istnieje element $a^{-1} \in G$, taki że:

$$a * a^{-1} = a^{-1} * a = e.$$

2.2 Grupy abelowe

Szczególnym przypadkiem grup są *grupy abelowe*, w których dodatkowo zachodzi własność przemienności działania:

$$a * b = b * a \quad \forall a, b \in G.$$

2.3 Przykłady grup

W celu zilustrowania wprowadzonych definicji rozważono kilka przykładów grup skończonych, które zostały następnie sformalizowane w systemie Rocq.

Grupa $(Z_4, +_4)$ została zdefiniowana jako czteroelementowy zbiór:

$$Z_4 = \{0, 1, 2, 3\},$$

wraz w działaniem dodawania modulo 4.

Podgrupa $Z_2 \leq Z_4$ W ramach grupy $(Z_4, +_4)$ można wyszczególnić podgrupę $(Z_2, +_4)$

$$Z_2 = \{0, 2\}.$$

z dodawaniem modulo 2.

Grupa $(Z_3, +_3)$ została zdefiniowana analogicznie do grupy $(Z_4, +_4)$:

$$Z_3 = \{0, 1, 2\}$$

z dodawaniem modulo 3.

3 Opis kodu

W ramach projektu wszystkie definicje i twierdzenia zostały zaimplementowane w jednym pliku `Groups.v`. Na początku wprowadzono definicje wykorzystywane w późniejszych twierdzeniach i przykładach.

3.1 Definicje

Na początku wprowadzono ogólną definicję grupy, która formalizuje aksjomaty grupy: łączność działania, istnienie elementu neutralnego oraz istnienie elementów odwrotnych.

Definition `is_group`

`(G : Type)`

```

(e : G)
(op : G -> G -> G)
(inv : G -> G)
: Prop :=
(forall a b c : G, op a (op b c) = op (op a b) c)
/\
(forall a : G, op e a = a)
/\
(forall a : G, op a e = a)
/\
(forall a : G, op (inv a) a = e)
/\
(forall a : G, op a (inv a) = e).

```

Wprowadzono ogólną definicję podgrupy.

Definition is_subgroup

```

(G : Type)
(e : G)
(op : G -> G -> G)
(inv : G -> G)
(H : Ensemble G)
: Prop :=
In G H e
/\
(forall a b : G, In G H a -> In G H b -> In G H (op a b))
/\
(forall a : G, In G H a -> In G H (inv a)).

```

Zdefiniowano lewostronną i prawostronną warstwę.

Definition left_coset

```

(G : Type)
(op : G -> G -> G)
(H : Ensemble G)
(g : G)
: Ensemble G := fun x : G => exists h : G, In G H h /\ x = op g h.

```

Definition right_coset

```

(G : Type)
(op : G -> G -> G)

```

```

(H : Ensemble G)
(g : G)
: Ensemble G := fun x : G => exists h : G, In G H h /\ x = op h g.

```

Następnie został zaimplementowany przykład, który polegał na sprawdzeniu, że $(Z_4, +_4)$ jest grupą oraz $(\{0, 2\}, +_4)$ jest podgrupą $(Z_4, +_4)$. W tym celu na początku został wprowadzony odpowiedni typ i działania.

```

Inductive Z4 : Type :=
  | z0 | z1 | z2 | z3.

```

```

Definition Z4_add (a b : Z4) : Z4 :=

```

```

Definition Z4_zero : Z4 := z0.

```

```

Definition Z4_inv (a : Z4) : Z4 :=

```

Kolejnym krokiem było udowodnienie pomocniczych lematów potrzebnych do wykazania, że $(Z_4, +_4)$ jest grupą.

```

Lemma Z4_add_zero_r :
  forall a, Z4_add a Z4_zero = a.

```

```

Lemma Z4_add_assoc :
  forall a b c, Z4_add a (Z4_add b c) = Z4_add (Z4_add a b) c.

```

```

Lemma Z4_inv_l :
  forall a, Z4_add (Z4_inv a) a = Z4_zero.

```

```

Lemma Z4_inv_r :
  forall a, Z4_add a (Z4_inv a) = Z4_zero.

```

Treść twierdzenia, że $(Z_4, +_4)$ jest grupą została zapisana w następujący sposób.

```

Theorem Z4_is_group :
  is_group Z4 Z4_zero Z4_add Z4_inv.

```

Następne dwa fragmenty kodu opisują podgrupę $(\{0, 2\}, +_4)$.

```

Definition Z2_subgroup : Ensemble Z4 :=
  fun x => x = z0 \/ x = z2.

```

```

Theorem Z2_is_subgroup :
  is_subgroup Z4 Z4_zero Z4_add Z4_inv Z2_subgroup.

```

3.2 Twierdzenia

Następnym naturalnym etapem jest sformułowanie i udowodnienie podstawowych twierdzeń z teorii grup. Ze względu na obszerność dowodów twierdzeń zamieszczono tylko sformułowania twierdzeń w Rocq, pominięto natomiast dowody twierdzeń. Wszystkie twierdzenia i dowody można znaleźć w pliku `Groups.v`.

3.2.1 Jednoznaczność elementu neutralnego w grupie

W każdej grupie jest dokładnie jeden element neutralny. To twierdzenie można zapisać w następujący sposób.

```
Theorem identity_unique_in_group :  
  forall (G : Type) (op : G -> G -> G) (e1 e2 : G),  
    is_identity G op e1 -> is_identity G op e2 -> e1 = e2.
```

Powyższe twierdzenie wykorzystuje `is_identity`, zdefiniowane w następujący sposób.

```
Definition is_identity  
  (G : Type)  
  (op : G -> G -> G)  
  (e : G)  
  : Prop := (forall a : G, op e a = a) /\ (forall a : G, op a e = a).
```

3.2.2 Element odwrotny

W każdej grupie każdy element ma jednoznacznie wyznaczony element odwrotny.

```
Theorem inverse_unique_in_group :  
  forall (G : Type) (e : G) (op : G -> G -> G) (inv : G -> G) (a b c : G),  
    is_group G e op inv -> is_inverse G e op a b -> is_inverse G e op a c -> b = c.
```

To twierdzenie wymaga wcześniejszego zdefiniowania `is_inverse`.

```
Definition is_inverse  
  (G : Type)  
  (e : G)  
  (op : G -> G -> G)  
  (a b : G)  
  : Prop := op a b = e /\ op b a = e.
```

3.2.3 Prawa skracania

W ramach projektu zapisano i udowodniono dwa twierdzenia opisujące prawa skracania. Dla dowolnych elementów $a, b, c \in G$ zachodzą implikacje:

$$a * b = a * c \Rightarrow b = c.$$

$$b * a = c * a \Rightarrow b = c.$$

Implementacja tych twierdzeń przedstawia się następująco.

Theorem left_cancellation :

```
forall (G : Type) (e : G) (op : G -> G -> G) (inv : G -> G) (a b c : G),
  is_group G e op inv -> op a b = op a c -> b = c.
```

Theorem right_cancellation :

```
forall (G : Type) (e : G) (op : G -> G -> G) (inv : G -> G) (a b c : G),
  is_group G e op inv -> op b a = op c a -> b = c.
```

3.2.4 Odwrotność elementu

Złożenie dwóch odwrotności jest identycznością. Można to twierdzenie zapisać równoważnie. Dla każdego elementu z grupy odwrotność od odwrotności od tego elementu to ten element.

Theorem inv_involutive :

```
forall (G : Type) (e : G) (op : G -> G -> G) (inv : G -> G) (a : G),
  is_group G e op inv -> inv (inv a) = a.
```

3.2.5 $(ab)^{-1} = b^{-1}a^{-1}$

W każdej grupie $(G, *)$ zachodzi następująca własność dotycząca elementów odwrotnych:

$$(a * b)^{-1} = b^{-1} * a^{-1}, \quad \forall a, b \in G.$$

Theorem inv_op :

```
forall (G : Type) (e : G) (op : G -> G -> G) (inv : G -> G) (a b : G),
  is_group G e op inv -> inv (op a b) = op (inv b) (inv a).
```

3.3 Grupy abelowe

Szczególną klasę grup stanowią *grupy abelowe*. Są to grupy, w których działanie grupowe spełnia dodatkowo własność przemienności, to jest dla dowolnych elementów $a, b \in G$

zachodzi:

$$a * b = b * a.$$

Grupy abelowe zdefiniowano w następujący sposób.

Definition is_abelian_group

```
(G : Type)
(e : G)
(op : G -> G -> G)
(inv : G -> G)
: Prop := is_group G e op inv /\ (forall a b : G, op a b = op b a).
```

W następnym twierdzeniu udowodniono, że $(Z_3, +_3)$ jest grupą abelową. Wymagało to zdefiniowania typu Z_3 .

Inductive Z3 : Type := | z30 | z31 | z32.

Definition Z3_add (a b : Z3) : Z3 :=

```
match a, b with
| z30, x => x
| x, z30 => x
| z31, z31 => z32
| z31, z32 => z30
| z32, z31 => z30
| z32, z32 => z31 end.
```

Definition Z3_zero : Z3 := z30.

Definition Z3_inv (a : Z3) : Z3 :=

```
match a with
| z30 => z30
| z31 => z32
| z32 => z31
end.
```

Theorem Z3_abelian :

```
is_abelian_group Z3 Z3_zero Z3_add Z3_inv.
```

4 Wykorzystane taktyki

Podczas formalizacji przedstawionych twierdzeń i definicji wykorzystano system dowodzenia *Rocq*. W dowodach wykorzystano różne taktyki przedstawione na zajęciach takie jak

`intros`, `apply`, `rewrite`, `destruct`, `exists`, `split`, `reflexivity`. Wykorzystano również dodatkowe taktyki:

- **Ensemble** jest sposobem reprezentacji zbiorów w Rocq. Zbiór nie jest tu listą elementów, lecz wyrażeniem określającym przynależność. Rozważmy następujący przykład:

```
Definition singleton (G : Type) (a : G) : Ensemble G :=
  fun x => x = a.
```

Powyższa definicja opisuje zbiór zawierający dokładnie jeden element a . Oznacza to, że element x należy do tego zbioru wtedy i tylko wtedy, gdy $x = a$.

- **subst** to taktyka służąca do podstawiania równych sobie wyrażeń. Jeśli w kontekście występuje równość $a = b$, to **subst** automatycznie zastępuje wszystkie wystąpienia a przez b , upraszczając cel dowodu.
- **exact** jest taktyką kończącą dowód, używaną wtedy, gdy posiadamy już dokładnie wyrażenie spełniające cel. Jeśli w kontekście znajduje się założenie identyczne z celem, **exact** pozwala bezpośrednio zakończyć dowód, wskazując to założenie jako rozwiązanie.

W trakcie dowodów wielu twierdzeń wykorzystano oznaczenia $+$, $-$, $*$, które pojawiają się przy wykorzystaniu `split` lub `destruct`. Nie są taktyki, służą tylko do rozdzielenia podcelów w trakcie dowodu. Celem wykorzystania takich oznaczeń jest zwiększenie czytelności i przejrzystości dowodu.

5 podsumowanie

W ramach projektu dokonano formalizacji wybranych pojęć i twierdzeń z teorii grup w systemie Rocq. Zdefiniowano podstawowe struktury algebraiczne, takie jak grupa, podgrupa oraz warstwy lewo- i prawostronne. Wykorzystując formalne dowodzenie w programie Rocq, udowodniono klasyczne własności grup, między innymi jednoznaczność elementu neutralnego i elementu odwrotnego, prawa skracania oraz własności odwrotności elementów i iloczynów. Omówiono również grupy abelowe z wykorzystaniem przykładu grupy abelowej $(Z_3, +_3)$.

Projekt wymagał poznania dodatkowych taktyk.